Governors State University
OPUS Open Portal to University Scholarship

All Capstone Projects | Student Capstone Projects

Fall 2023

PatientCare Xpert

Abdul Qayyum Mohammed

Follow this and additional works at: https://opus.govst.edu/capstones

For more information about the academic degree, extended learning, and certificate programs of Governors State University, go to http://www.govst.edu/Academics/Degree_Programs_and_Certifications/

Visit the Governors State Computer Science Department
This Capstone Project is brought to you for free and open access by the Student Capstone Projects at OPUS Open Portal to University Scholarship. It has been accepted for inclusion in All Capstone Projects by an authorized administrator of OPUS Open Portal to University Scholarship. For more information, please contact opus@govst.edu.

PATIENTCARE XPERT

By

Abdul Qayyum Mohammed
Bachelor Of Engineering Civil Engineering
Osmania University Hyderabad,2018

GRADUATE CAPSTONE SEMINAR PROJECT

Submitted in partial fulfillment of the requirements

For the Degree of Master of Science,

With a Major in Computer Science

Governors State University
University Park, IL 60484

2023

ABSTRACT

The medical consultation web app is a comprehensive and secure platform designed to facilitate remote healthcare consultations. Developed using .NET Core 7 framework, the application leverages the latest technologies to provide a seamless and efficient medical consultation experience. The web app caters to patients, doctors, and medical professionals, allowing them to connect and conduct consultations remotely. The platform prioritizes privacy and data security, ensuring that all communication and medical information shared during the consultation is encrypted and protected. Key features of the web app include a user-friendly interface, secure login and authentication, appointment scheduling, secure messaging, patient analysis report and e-prescriptions. Users can easily find and connect with doctors specialized in various medical fields, view their profiles, and book appointments based on their availability. The web app utilizes the .NET Core 7 framework to ensure high performance, scalability, and stability. It also integrates with various medical databases and systems to access patient records securely, supporting comprehensive medical histories for accurate and informed consultations.

By enabling remote medical consultations, the application offers convenience and accessibility to patients, especially those who may have limited mobility or live in remote areas. It also improves efficiency for healthcare professionals by reducing the need for physical appointments and allowing them to provide timely and accessible healthcare services. At the scheduled time, the user initiates a video or audio call with the doctor through the app. They discuss their symptoms, share medical history, and ask questions. The application will utilize the MVC Framework for both backend and front-end development, while HTML, CSS, BOOTSTRAP, C# will be used for the user interface design. The data generated by the application will be stored in an MSSQL database.

In conclusion, the medical consultation web app built on .NET Core 7 framework offers a secure, efficient, and user-friendly platform for remote healthcare consultations. It aims to bridge the gap between patients and healthcare professionals, ensuring quality and accessible healthcare services for everyone.

Table of Content

1 Project Description 1
1.1 Competitive Information 1
1.2 Relationship to Other Applications/Projects 2
1.3 Assumptions and Dependencies 2
1.4 Future Enhancements 2
1.5 Definitions and Acronyms 3
2 Project Technical Description 3
2.1 Application Architecture 5
2.2 Application Information flows 8
2.3 Interactions with other Applications 9
2.4 Capabilities 10
2.5 Risk Assessment and Management 10
3 Project Requirements 11
3.1 Identification of Requirements 11
3.2 Operations, Administration, Maintenance and Provisioning (OAM&P) 12
3.3 Security and Fraud Prevention 12
3.4 Release and Transition Plan 13
4 Project Design Description 13
5 Internal/external Interface Impacts and Specification 14
6 Design Units Impacts 21
6.1 Functional Area A/Design Unit A 22
6.1.1 Functional Overview 22
6.1.2 Impacts 22
6.1.3 Requirements 22
7 Open Issues 22
8 Acknowledgements 23
9 References 23
10 Appendices 23

1 Project Description

The main objective of PatientCare Xpert is to revolutionize the healthcare experience by providing a comprehensive and user-centric platform for patients, caregivers, and healthcare professionals. This innovative project is designed to address key challenges in the healthcare industry, focusing on enhancing patient care, improving accessibility to medical services, and streamlining communication among stakeholders. PatientCare Xpert aims to create a seamless ecosystem where patients can easily manage their health records, schedule appointments, and engage in remote consultations with healthcare professionals. The primary goal is to empower patients with tools that facilitate proactive healthcare management and enable timely interventions. Through an intuitive and secure interface, PatientCare Xpert strives to foster efficient communication between patients and healthcare providers. The project emphasizes privacy and data security, ensuring that sensitive medical information is protected throughout the patient journey. By leveraging cutting-edge technologies and adhering to industry standards, PatientCare Xpert aims to set a new benchmark for the delivery of healthcare services. Key features of PatientCare Xpert include appointment scheduling, secure messaging, remote consultation capabilities, and a centralized repository for medical records. The project also incorporates user-friendly interfaces for both patients and healthcare professionals, promoting a positive and engaging experience. In conclusion, the main objective of PatientCare Xpert is to reshape the healthcare landscape, placing the patient at the center of the care continuum. This project seeks to improve health outcomes, increase patient satisfaction, and contribute to the overall advancement of healthcare delivery through innovative technology and a patient-centric approach.

1.1 Competitive Information

In the competitive landscape, PatientCare Xpert stands as a strong contender alongside Medicare360 and HealthConnectPro (ProHealthConnect n.d.), each bringing distinctive strengths to the healthcare technology sector. While Medicare360 excels in offering an end-to-end healthcare management system, PatientCare Xpert distinguishes itself through a user-centric model and a comprehensive approach to patient care. PatientCare Xpert's emphasis on remote consultations, secure messaging, and proactive healthcare management positions it as a dynamic and innovative solution, complementing Medicare360's strengths. Similarly, in comparison to HealthConnectPro, PatientCare Xpert stands out with its intuitive user interface, emphasizing privacy and data security. While HealthConnectPro is recognized for its interoperability and collaborative features, PatientCare Xpert's commitment to seamless patient

experience and integrated functionalities adds a layer of competitiveness to the market. Overall, PatientCare Xpert aims to offer a robust alternative with a focus on patient empowerment, data security, and a multifaceted approach to healthcare technology.

1.2 Relationship to Other Applications/Projects

In the realm of healthcare technology, PatientCare Xpert maintains a strategic relationship with other applications and projects to foster a cohesive and interoperable healthcare ecosystem. Its integration points are carefully designed to align with existing healthcare systems, ensuring seamless data flow and collaboration. PatientCare Xpert prioritizes adherence to industry standards and regulatory compliance, guaranteeing compatibility and data security. This interoperability extends to third-party services, enabling scalability and resilience. Ultimately, PatientCare Xpert's relationship with other applications and projects is characterized by a commitment to enhancing overall healthcare efficiency, user experience, and regulatory adherence within the interconnected healthcare landscape.

1.3 Assumptions and Dependencies

- PatientCare Xpert assumes compatibility with current and upcoming versions of .NET Core 7, HTML5, CSS, Bootstrap, and C# for both backend and frontend development.
- Assumption is made that users will have stable internet connectivity for seamless access to remote consultations and data synchronization.
- Dependencies on regulatory compliance standards and any updates made to these standards during the project development.
- Assumes that healthcare professionals will actively adopt and engage with the platform for remote consultations and patient management.
- Dependencies exist on the continuous availability and reliability of third-party services used, such as signalR for peering video conferencing.
- Dependency on the implementation of secure data transmission protocols to safeguard patient information during remote consultations.
- Assumes that the architecture and infrastructure are scalable to accommodate potential increases in user volume and data.

1.4 Future Enhancements

One key area of focus is the implementation of advanced analytics and predictive insights. By leveraging sophisticated data analytics techniques, PatientCare Xpert aims to empower healthcare professionals with predictive capabilities, enabling proactive decision-making and personalized treatment strategies. Additionally, PatientCare Xpert is exploring the integration of wearable devices into its ecosystem. This future enhancement seeks to facilitate real-time monitoring of patients' health metrics, fostering a more

comprehensive and continuous approach to remote healthcare. The platform also envisions further evolution in its telemedicine features, potentially incorporating augmented reality (AR) for enhanced visual diagnostics and enriched virtual healthcare experiences. This expansion aligns with the growing demand for immersive and interactive healthcare solutions. Artificial intelligence (AI) will play a pivotal role in PatientCare Xpert's future, with plans to develop AI-driven personalized health plans. These plans will leverage individual health data and preferences to offer tailored and proactive health management strategies. To enhance accessibility, PatientCare Xpert plans to ensure cross-platform availability, including dedicated applications for various mobile devices and additional web browsers. This strategic move aims to broaden the reach of the platform, making it more accessible to a diverse user base. Lastly, the introduction of automated appointment reminders and scheduling optimization algorithms is on the horizon. This enhancement aims to streamline appointment management, reduce no-show rates, and enhance the overall efficiency of healthcare service delivery.

1.5 Definitions and Acronyms

- LINQ (Language Integrated Query) - A set of features in C# that provides a unified approach to querying and manipulating data.
- Entity Framework (EF) - An object-relational mapping (ORM) framework for .NET that simplifies database interactions by enabling developers to work with databases using .NET objects.
- MSSQL (Microsoft SQL Server) - A relational database management system (RDBMS) developed by Microsoft, widely used for storing and retrieving data.
- AJAX (Asynchronous JavaScript and XML) - A set of web development techniques that allows web pages to be updated asynchronously by exchanging small amounts of data with the server behind the scenes.
- UI (User Interface) - The point of interaction between the user and a computer application, encompassing elements like screens, pages, and graphical elements.

2 Project Technical Description

Within PatientCare Xpert, a robust technical framework is harnessed to offer specialized features for healthcare specialists, an inviting home experience, and convenient access to available slots for visitors without the need for registration. Developed on .NET Core 7, this platform integrates cutting-edge technologies, including SignalR for real-time communication, LINQ statements for efficient data querying, peering video functionality for enhanced remote consultations, and a responsive user interface

crafted with CSS3, HTML5, Bootstrap 5, and JavaScript. The underlying MSSQL database ensures secure and reliable data storage. PatientCare Xpert encompasses distinct roles patient, admin, and doctor each offering specialized functionalities, coupled with an intuitive user experience for visitors without registration. Patients accessing PatientCare Xpert experience a user-friendly interface that enables them to schedule remote consultations, view analysis reports, and manage their health records securely. The platform caters to their needs by offering personalized health plans, telemedicine features, and real-time communication with healthcare professionals. The user-centric design ensures a seamless experience, fostering active patient engagement. Administrators wield powerful capabilities within PatientCare Xpert, overseeing system configurations, user management, and ensuring regulatory compliance. Admins have the authority to manage departments, blood group, and oversee the overall system's stability. Their role is pivotal in maintaining data integrity and ensuring a secure healthcare environment for patients and healthcare professionals. Healthcare professionals accessing PatientCare Xpert find a tailored interface that facilitates remote consultations, patient analysis, and prescription management. The platform empowers doctors with tools for real-time collaboration, secure messaging, and access to patient records. The user experience is designed to streamline workflows, enhancing the efficiency of healthcare delivery. For visitors exploring PatientCare Xpert without registration, the platform provides a glimpse into specialized services, home page offerings, and available consultation slots. Visitors can view information about healthcare specialists, explore the platform's features, and check appointment availability. This approach ensures that even without registration, visitors gain valuable insights into the platform's capabilities, fostering interest and engagement.

The seamless integration of video conferencing using SignalR elevates the healthcare experience for both patients and doctors. Through the implementation of a dedicated SignalR hub, the platform facilitates real-time bidirectional communication, ensuring secure and authenticated access for all users. The appointment scheduling feature adds a layer of convenience, allowing patients and doctors to connect effortlessly at the scheduled time. Behind the scenes, the system employs WebRTC technology for establishing peer-to-peer connections, enabling the smooth streaming of audio and video during appointments. Real-time signaling mechanisms handle the initiation, acceptance, and termination of video calls, while intuitive interfaces make the entire process user-friendly. Furthermore, robust error handling and bandwidth optimization contribute to a reliable and high-quality video conferencing experience. Altogether, PatientCare Xpert leverages SignalR and cutting-edge technologies to redefine healthcare communication, making virtual appointments as seamless and effective as in-person consultations.

```
        }
        [Authorize(Roles = "Admin,Patient,Doctor")]
        0 references
        public IActionResult _QuickAction() {
            ViewBag.Patient = _context.Users.Where(u => u.Role == "Patient").Count();
            ViewBag.Doctor = _context.Users.Where(u => u.Role == "Doctor").Count();
            ViewBag.Department = _context.departments.Count();
            ViewBag.Medicine = _context.medicines.Count();
            ViewBag.AvbSlots = _context.doctorSlots.Where(avb=>avb.Slotstatus == slots.Available && avb.StartTime > DateTime.Now).Count();
            var appointmnet = _context.appointments.Where(app => app.AppointmentStatus == "B").ToList();
            var appointmnetpaid = appointmnet.Join(_context.payments, ap => ap.Id, p => p.AppointmentId, (ap, p) => new { ap }).ToList();
             ViewBag.Payments = appointmnetpaid.Sum(ap => ap.ap.VisitCharges);
            var loginUserval = this.User.FindFirstValue(ClaimTypes.NameIdentifier);

            if(User.IsInRole("Doctor")) ViewBag.DoctorPayments = appointmnetpaid.Where(ap=>ap.ap.DoctorId == loginUserval).Sum(ap => ap.ap.VisitCharges);
            if (User.IsInRole("Patient")) ViewBag.PaymentSettled = appointmnetpaid.Where(ap => ap.ap.PatientId == loginUserval).Sum(ap => ap.ap.VisitCharges);
            return View();
        }
        [Authorize(Roles = "Admin")]
        0 references
        public IActionResult Department(AdminViewModel model, int? depid)
        {

            if (Request.Method == "GET")
            {
                model.departmentList = _adminInformation.GetDepartments();
                model.department = depid > 0 ? _adminInformation.GetDepartment((int)depid) : new Department();
                return View(model);
            }
            var dep = _adminInformation.AddUpdateDepartment(model.department);
            if (!dep.IsNullOrEmpty())
            {
                return RedirectToAction("Department", depid = 0);
            }
            return View(model);
        }
        [Authorize(Roles = "Admin")]
        0 references
        public IActionResult Delete(int depid)
        {
            var val = _adminInformation.DeleteDepartment(depid);
            if (val == 1)
                return RedirectToActionPermanent("Department");
            else
                return View("Department", new { depid = depid });
        }
        [Authorize(Roles = "Admin")]
        0 references
        public IActionResult BloodGroup(AdminViewModel model, int? bgid)
        {

            if (Request.Method == "GET")
            {
                model.bgList = _adminInformation.GetBloodGroups();
```

Figure 1 Code snippet of Home Controller

2.1 Application Architecture

PatientCare Xpert is meticulously designed based on the Model-View-Controller (MVC) architecture, providing a robust foundation for efficient code organization and scalability. This architectural pattern separates the application into three interconnected components:

Model (M): The Model encapsulates the application's data layer and business logic. PatientCare Xpert relies on a Microsoft SQL Server (MSSQL) database, ensuring structured and relational data storage. The Model handles operations related to patient records, medical history, appointments, and other crucial data aspects.

View (V): Views in PatientCare Xpert are responsible for presenting a user-friendly interface. The application embraces responsive web design principles, offering an intuitive experience across various devices. The user interface includes the patient dashboard, appointment scheduling screens, and the dedicated interface for video conferencing.

Controller (C): Controllers act as intermediaries, managing user inputs and orchestrating the interaction between the Model and the View. They govern the flow of data within the application. PatientCare Xpert leverages ASP.NET MVC controllers to handle HTTP requests and shape the application's behavior.

PatientCare Xpert's application architecture is designed to seamlessly manage the entire healthcare workflow, encompassing functionalities for administrators, doctors, and patients. Leveraging Entity Framework (EF) migrations, the platform ensures a dynamic and evolving database structure. The application employs EF migrations to facilitate the evolution of the database structure. Admins play a pivotal role in this process, creating and managing essential entities such as departments, blood groups,

and doctor entries. The dynamic nature of EF migration ensures the adaptability of the database to changing requirements. Doctors access the system with unique credentials, allowing them to create and manage their available slots. Admin-created departments and blood groups enhance the doctor's profile, providing comprehensive information to patients during the appointment booking process. Patients register on the platform, initiating the appointment booking process. The intuitive interface allows patients to view available slots, select a doctor, and request an appointment. Notably, appointments can only be booked after doctor approval, ensuring a streamlined and coordinated scheduling process. Upon receiving a patient's appointment request, doctors review and approve appointments. Subsequently, doctors have the capability to fix charges for specific patients based on the nature of the appointment. This ensures transparency and facilitates a clear understanding of associated costs. A standout feature of PatientCare Xpert is its integration of SignalR peering video functionality. This innovative technology allows doctors and patients to engage in real-time video consultations. Doctors may choose to provide an external Zoom meeting link or utilize the platform's integrated video functionality. The "Join Meeting" button facilitates a seamless connection, enabling doctors and patients to interact remotely. This comprehensive application architecture ensures an efficient, secure, and user-friendly healthcare experience for administrators, doctors, and patients alike. The incorporation of EF migrations, dynamic appointment workflows, and advanced video functionality aligns PatientCare Xpert with the latest trends in healthcare technology, fostering a collaborative and streamlined approach to remote healthcare consultations. PatientCare Xpert incorporates a cutting-edge video conferencing feature facilitated by SignalR, offering seamless and real-time communication between patients and healthcare providers. The system utilizes a peering model, ensuring direct and secure connections for efficient video consultations. The heart of the video conferencing functionality lies in the SignalR Hub dedicated to managing real-time communication. This hub orchestrates the initiation and acceptance of video calls between patients and doctors. PatientCare Xpert leverages WebRTC (Web Real-Time Communication) for peer-to-peer communication during video conferences. WebRTC ensures secure and direct data exchange, enabling high-quality video and audio streaming between the patient and the doctor. By integrating SignalR with a peering model, PatientCare Xpert delivers a robust and responsive video conferencing experience, fostering effective virtual consultations, and enhancing the overall telemedicine capabilities of the platform.

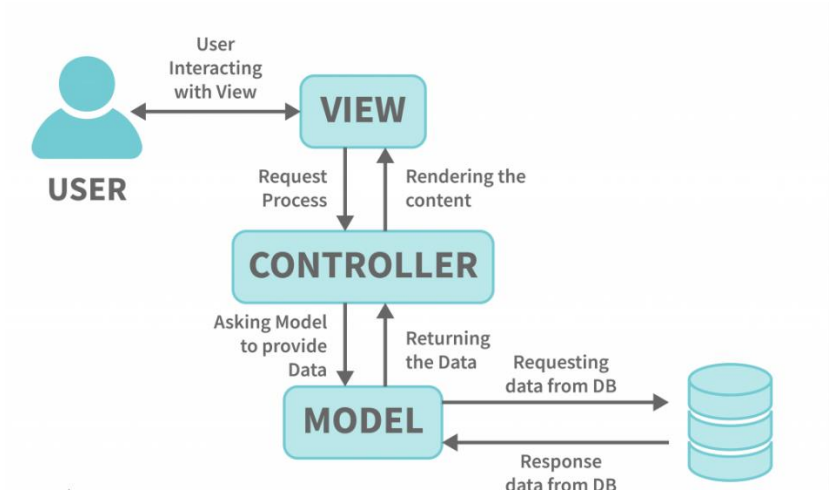


Figure 2 Application Architecture of PatientCare Xpert (MVC Architecture n.d.)

Code Snippets:

```
namespace PatientcareXpert.Models.Hubs
{
    1 reference
    public class DefaultHub :Hub
    {
        0 references
        public async Task JoinRoom(string roomId, string userId)
        {
            Users.list.Add(Context.ConnectionId, userId);
            await Groups.AddToGroupAsync(Context.ConnectionId, roomId);
            await Clients.Group(roomId).SendAsync("user-connected", userId);
        }

        0 references
        public override Task OnDisconnectedAsync(Exception? exception)
        {
            Clients.All.SendAsync("user-disconnected", Users.list[Context.ConnectionId]);
                return base.OnDisconnectedAsync(exception);
        }
        0 references
        public async Task LeaveRoom(string roomName)
        {
            //await Groups.RemoveFromGroupAsync(Context.ConnectionId, roomName);
            await Clients.Group(roomName).SendAsync("UserLeft", Context.ConnectionId);
        }

        0 references
        public async Task SendMessage(string roomName, string message)
        {
            await Clients.Group(roomName).SendAsync("ReceiveMessage", Context.ConnectionId, message);
        }
    }
```

Figure 3 SignalR

```
@{
    ViewData["Title"] = "Patient Doctor Discussion";
    Layout = "_SecondaryLayout";
}
@*<link href="~/css//streaming.css"/>*@
<div class="container-xxl flex-grow-1 container-p-y">
    <h1 hidden>@ViewBag.roomId</h1>
    <h4 class="text-center">Patient and Doctor Discussion </h4>
    <div video-grid></div>
    <div class="col-md-3">
        <a href="javascript:window.open('','_self').close();" class="btn btn-danger">Leave Meeting</a>
    </div>
</div>
<script src="~/lib/microsoft/signalr/dist/browser/signalr.min.js"></script>
<script src="https://unpkg.com/peerjs@1.3.2/dist/peerjs.min.js"></script>
<script>
    const ROOM_ID = '@ViewBag.roomId'
    let userId = null
    let localStream = null
    const Peers = {}
    const connection = new signalR.HubConnectionBuilder()
        .withUrl("/meeting")
        .build();
    const myPeer = new Peer()
    myPeer.on('open', id => {
        userId = id;

        const startSignalR = async () => {
            await connection.start();
            await connection.invoke("JoinRoom", ROOM_ID, userId)
```

Figure 4 View Page

2.2 Application Information flows

PatientCare Xpert's architecture orchestrates a cohesive information flow, ensuring seamless interactions between administrators, doctors, and patients. The application's processes are designed to optimize data exchange and enhance user experiences throughout the healthcare journey. Administrators initiate the information flow by configuring vital elements such as departments, blood groups, and doctor entries through the EF migration framework. This foundational setup ensures a dynamic and structured database, laying the groundwork for subsequent operations. Upon accessing the system with unique credentials, doctors contribute to the information flow by managing their profiles. They input critical details, including available slots, which seamlessly update the database. This dynamic update ensures that the platform reflects the latest information about a doctor's availability. Patients engage with the system by registering, and their information seamlessly integrates into the database. The patient-driven flow continues as they explore available slots, select a doctor, and request an appointment. This request is then transmitted to the doctor for approval, marking the beginning of the appointment coordination process. Doctors participate in the information flow by reviewing and acting on patient appointment requests. Upon approval, doctors have the ability to fix charges for the specific patient, enriching the appointment details stored in the database. This dynamic interaction ensures transparency in the appointment process. A distinctive feature of PatientCare Xpert is the SignalR peering video functionality. This technology facilitates real-time video consultations between doctors and patients. Doctors, upon approval of an appointment, can choose to provide an external Zoom meeting link or utilize the integrated video functionality within the platform. Patients seamlessly join the video consultation through the "Join Meeting" button.

Real-time updates on appointment statuses, including approvals and rejections, are communicated to both doctors and patients. This immediate feedback ensures transparency, allowing stakeholders to stay informed about any changes in the appointment status. If a doctor opts to provide an external Zoom meeting link, this information is effortlessly shared with the patient. The link serves as the gateway for both parties to connect remotely for the scheduled appointment, enhancing the accessibility of healthcare services. To maintain a comprehensive record of interactions, the application logs appointment histories, charges, and communication between doctors and patients. This database-driven approach ensures that stakeholders can access a detailed history of their engagements within the platform.

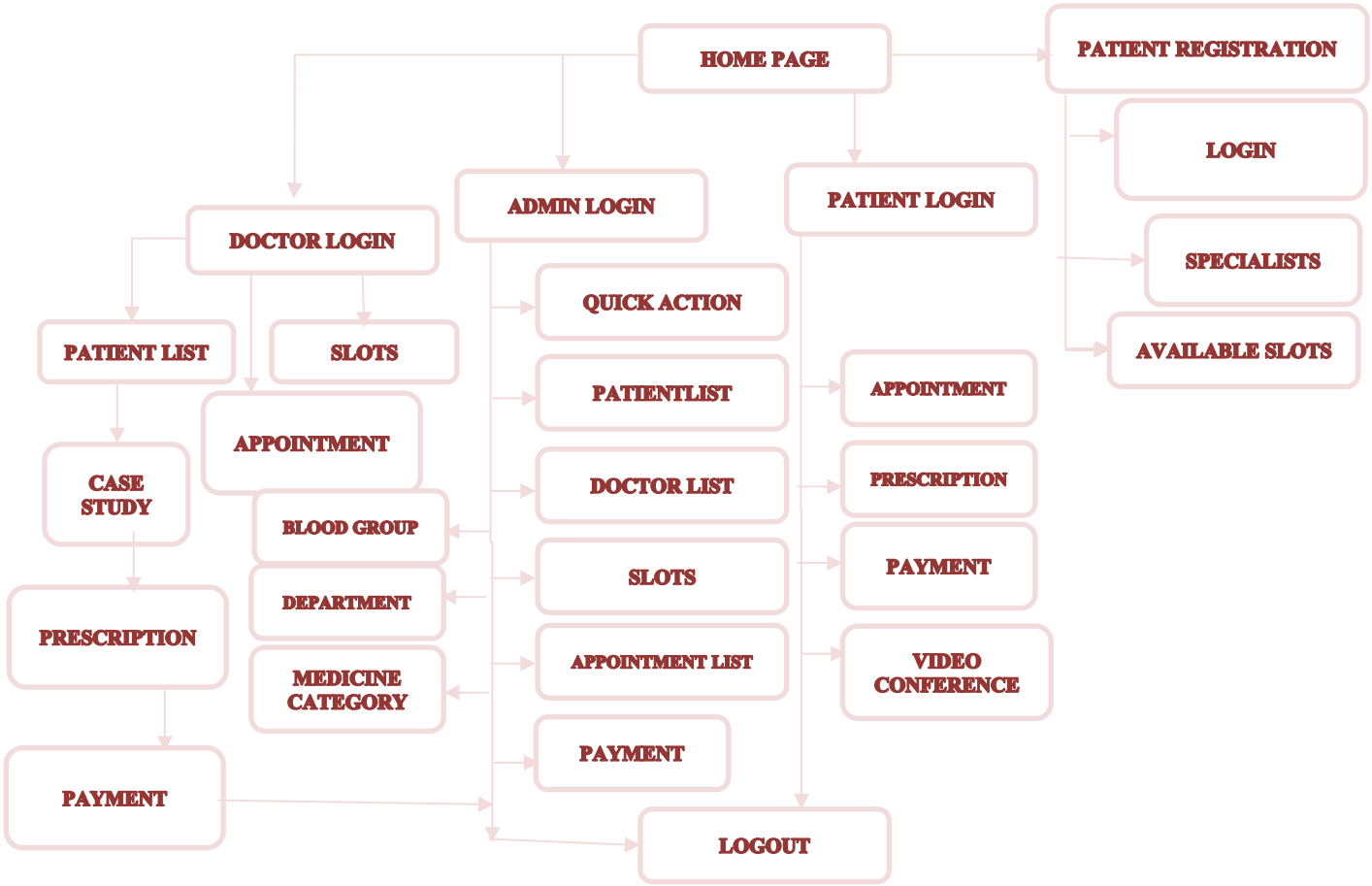


Figure 5 Workflow Diagram

2.3 Interactions with other Applications

PatientCare Xpert is not an isolated healthcare solution; it thrives on seamless interactions with various applications, enhancing its capabilities and providing an all-encompassing healthcare experience. The platform's strategic integration and collaboration with external applications contribute to optimized data exchange, efficient communication, and a holistic healthcare ecosystem. One pivotal integration is with the Zoom application, a widely used platform for virtual communication. PatientCare Xpert leverages this integration to enhance remote video consultations. When doctors opt to provide an external Zoom meeting link for appointments, the application seamlessly synchronizes with Zoom. This integration ensures a robust and trusted platform for secure and effective video communication between patients and healthcare professionals. SignalR, a real-time communication library, is another key player in PatientCare Xpert's interactions (SignalR n.d.). The platform utilizes SignalR's peering video technology to facilitate responsive and interactive video consultations. This technology is seamlessly embedded within the PatientCare Xpert platform, fostering real-time communication channels that enhance the user experience for both doctors and patients. The backbone of data management within PatientCare Xpert lies in its extensive interaction with the MSSQL database. The application utilizes the Entity Framework (EF)

migration framework to ensure efficient management of the database. Interactions with the database are pervasive, occurring during crucial stages such as patient registration, appointment booking, doctor profile management, and recording interaction logs. This strategic integration ensures a structured and resilient data storage system, providing a solid foundation for the entire healthcare workflow. PatientCare Xpert also facilitates the sharing of external Zoom meeting links when doctors choose this option for appointments. The platform ensures the seamless communication of these links to patients, enabling them to join scheduled Zoom meetings directly from the platform. This streamlined process enhances the accessibility of remote consultations, providing a user-friendly experience for both patients and doctors. This framework provides a solid foundation for the platform's architecture, ensuring compatibility, scalability, security, and performance. The integration with .NET Core 7 aligns PatientCare Xpert with the latest advancements in the .NET ecosystem, contributing to its technological prowess.

2.4 Capabilities

- ✓ Efficient management of user roles, including administrators, doctors, and patients.
- ✓ Role-specific dashboards and functionalities tailored to the needs of each user category.
- ✓ Utilization of EF migration for dynamic and seamless updates to the database structure.
- ✓ Structured storage and retrieval of critical data such as patient records, appointments, and doctor profiles.
- ✓ Patient-initiated appointment requests, subject to doctor approval.
- ✓ Doctors can fix charges for specific patients.
- ✓ Integration of SignalR peering video technology for responsive and interactive video consultations.
- ✓ An option for doctors to provide an external Zoom meeting link for remote appointments.
- ✓ Seamless integration with the Zoom application for enhanced video communication.
- ✓ Effortless sharing of external Zoom meeting links for scheduled appointments.
- ✓ Doctors can fix charges for specific patients, ensuring transparency in healthcare costs.
- ✓ Integrated charge fixing within the appointment workflow.
- ✓ Maintenance of detailed interaction logs, including appointment history and communication.

2.5 Risk Assessment and Management

A comprehensive risk assessment and management strategy has been integrated into the development and operational processes to ensure the platform's robustness, security, and reliability. The platform is continuously monitored for emerging risks, and the risk assessment is regularly updated to reflect changes

in the healthcare landscape and technological advancements. The team remains vigilant to evolving threats and employs agile methodologies to adapt the platform to emerging challenges effectively. Effective risk management extends to collaborative efforts with stakeholders, including administrators, doctors, and patients. Transparent communication channels are established to report and address any concerns or issues promptly. Stakeholder involvement ensures that potential risks are identified and mitigated collectively, fostering a sense of shared responsibility for the platform's success. PatientCare Xpert's risk assessment and management strategy form a crucial foundation for the platform's reliability and resilience. By embracing a proactive and collaborative approach, the platform not only identifies and mitigates potential risks but also ensures continuous improvement and adaptation to the evolving healthcare landscape. This commitment to risk management underscores PatientCare Xpert's dedication to providing a secure and trustworthy healthcare management solution.

3 Project Requirements

3.1 Identification of Requirements

<GSU-PCX_2023-01 Admin-capability-PCX001>

Settings - Configuration of system settings, including departments, blood groups, and doctor entries. Management of user roles, permissions, and overall system configurations

< GSU-PCX_2023-02 Doctor-capability-PCX002>

Manage - Access to a dedicated dashboard for managing available slots, appointments, and patient interactions.

< GSU-PCX_2023-03 Doctor-capability-PCX003>

Consultation - Ability to provide external Zoom meeting links for remote consultations.

< GSU-PCX_2023-04 Patient-capability- PCX004>

Registration - User-friendly registration process, allowing patients to create accounts and manage personal information.

< GSU-PCX_2023-05 Patient-capability-PCX005>

Appointment- Booking and managing appointments with doctors, including appointment approval and charge fixation.

< GSU-PCX_2023-06 VideoConsultation-capability-PCX006>

SignalR- Integration of SignalR peering video functionality for responsive and interactive video consultations.

3.2 Operations, Administration, Maintenance and Provisioning (OAM&P)

The Operations, Administration, Maintenance, and Provisioning (OAM&P) of PatientCare Xpert play a pivotal role in ensuring the seamless functioning, efficient management, and continuous improvement of the healthcare management platform.

Operations

Implement automated system checks to monitor the platform's health. Regularly assess the responsiveness and performance of critical functionalities. Track user interactions to identify patterns and potential issues.

Administration

Facilitate easy administration of user roles and permissions. Provide an intuitive dashboard for administrators to configure and manage user access. Ensure the flexibility to adapt configurations based on changing healthcare requirements.

Maintenance

Implement a schedule for regular system updates and patches. Ensure that security patches are applied promptly to address potential vulnerabilities. Conduct routine database maintenance to optimize performance. Regularly review and optimize database queries for efficiency.

Provisioning

Streamline the onboarding process for new users, including administrators, doctors, and patients. Provide training resources to ensure users are proficient in utilizing the platform. Develop provisions for the scalable expansion of the platform. Plan for increased user numbers, additional functionalities, and evolving healthcare needs.

3.3 Security and Fraud Prevention

Ensuring robust security measures and implementing fraud prevention strategies are critical aspects of PatientCare Xpert's commitment to safeguarding sensitive healthcare data and maintaining the integrity of the platform.

- Analyze user behavior patterns to identify anomalies or deviations.
- Implement automated systems to flag and investigate suspicious user activities.
- Enforce role-based access controls to restrict user permissions.
- Regularly review and update access control policies to align with evolving security needs
- Regularly back up critical data to prevent data loss in the event of security incidents.
- Store backups securely to maintain data integrity.

3.4 Release and Transition Plan

The Release and Transition Plan for PatientCare Xpert is designed to ensure a methodical and user-focused deployment. By prioritizing testing, user training, and phased deployment, PatientCare Xpert aims to deliver a secure and reliable healthcare management platform that meets the needs of administrators, doctors, and patients while allowing for continuous improvement post-launch. Facilitate the seamless migration of existing data to the live platform. Verify data integrity and accuracy during the transition process. Develop a comprehensive communication plan to inform stakeholders, including administrators, doctors, and patients, about the platform's release. Clearly articulate the benefits, features, and support channels available.

4 Project Design Description

The design of PatientCare Xpert is meticulously crafted to provide a comprehensive, user-friendly, and secure healthcare management platform. Prioritize a user-centric design to ensure a seamless experience for administrators, doctors, and patients. Implement intuitive navigation and visually appealing interfaces to enhance user engagement. Employ a responsive design approach to ensure optimal user experiences across various devices and screen sizes. Adapt the platform's layout dynamically based on the user's device. Design a scalable architecture that accommodates the platform's growth and evolving healthcare needs. Incorporate modular components to facilitate future enhancements and feature additions. Implement distinct dashboards for administrators, doctors, and patients, tailored to their specific roles.
Provide role-specific functionalities and information relevant to each user group. Develop a robust appointment management system allowing patients to schedule appointments, doctors to manage available slots, and administrators to oversee the overall scheduling process. Integrate SignalR peering video functionality for secure and interactive video consultations. Allow doctors to provide external Zoom meeting links for remote consultations. Utilize Entity Framework for efficient data storage, retrieval, and management. Implement secure data encryption to safeguard patient records and sensitive information. Integrate fraud prevention algorithms to detect and prevent potentially fraudulent activities. Analyze user behavior patterns to identify anomalies indicative of fraud. Doctors log in to their dedicated dashboard to manage available slots, view appointment requests, and approve appointments. The platform enables doctors to set charges for specific patients and provide external meeting links for remote consultations.

Patients register on the platform, book appointments with preferred doctors, and await approval. Once approved, patients pay charges and join video consultations using the provided links. Administrators have access to a centralized dashboard for overall system configuration. Manage user roles, configure system

settings, and oversee the entire healthcare management process. Developed using .NET Core 7 framework for enhanced performance, security, and scalability. Leveraged SignalR for real-time communication and video conferencing. Utilized HTML5, CSS3, Bootstrap 5, and JavaScript for responsive and dynamic front-end design. Employed MSSQL as the backend database for structured data storage.

5 *Internal/external Interface Impacts and Specification*

The internal and external interfaces of PatientCare Xpert play a crucial role in shaping the user experience and ensuring seamless interactions within the healthcare management platform. Different user roles (administrator, doctor, patient) have distinct dashboards with role-specific functionalities. Ensure role-based access controls, with each user accessing a dedicated dashboard aligned with their responsibilities. SignalR peering video functionality enables secure video consultations. Integrate SignalR for real-time communication and provide the option for doctors to share external Zoom meeting links.

Software Requirements:

Operating System	**:**	Windows
Languages	**:**	C#
Tools	**:**	Visual Studio Code
Database	**:**	Microsoft SQL

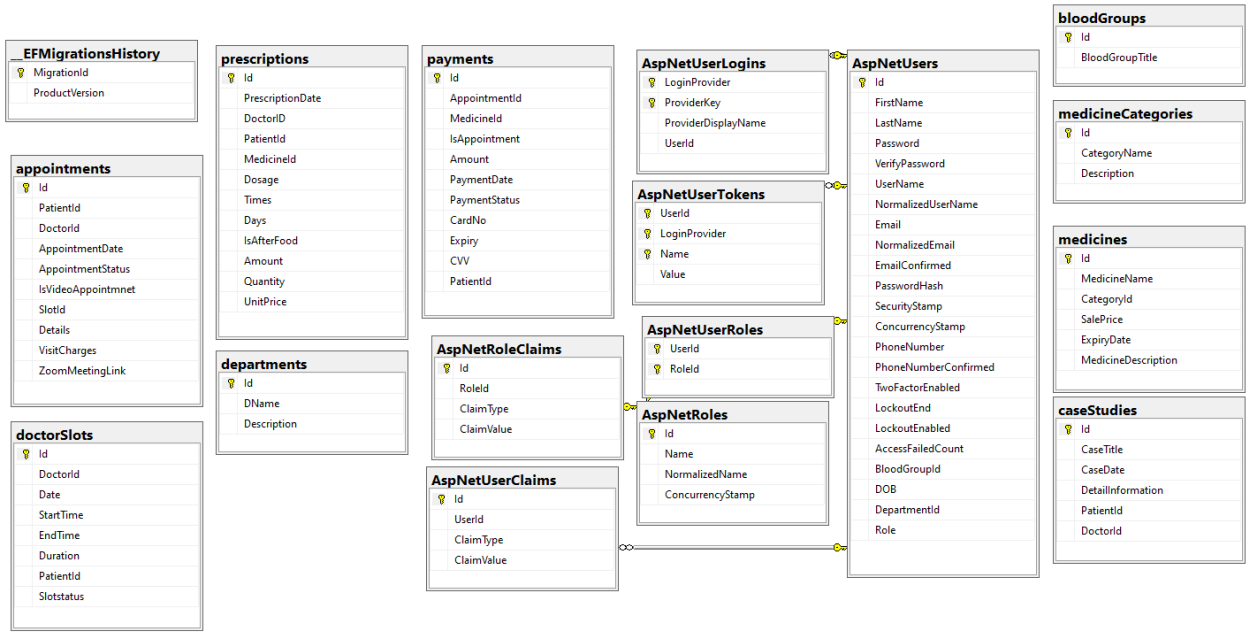


Figure 6 ER Diagram for Database

Figure 7 Software Development Process

The features of the application's design can be seen in the following screenshots:

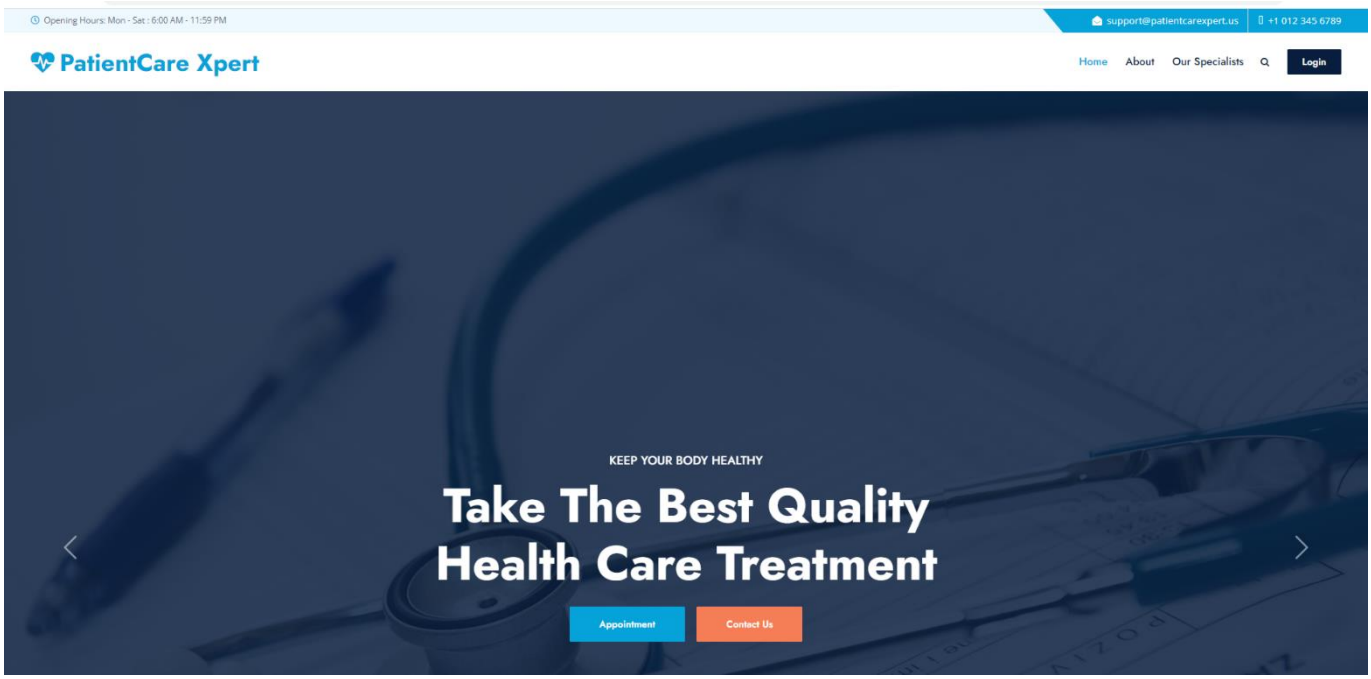


Figure 8 Main Page

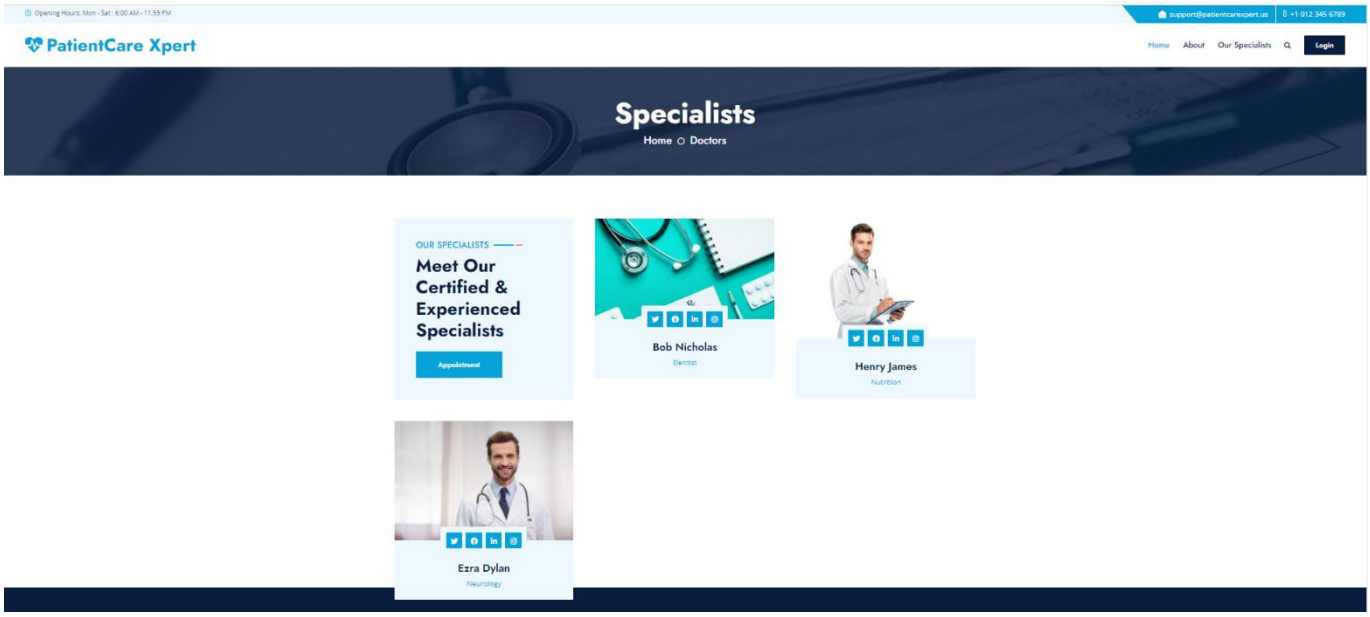


Figure 9 Our Specialists Page

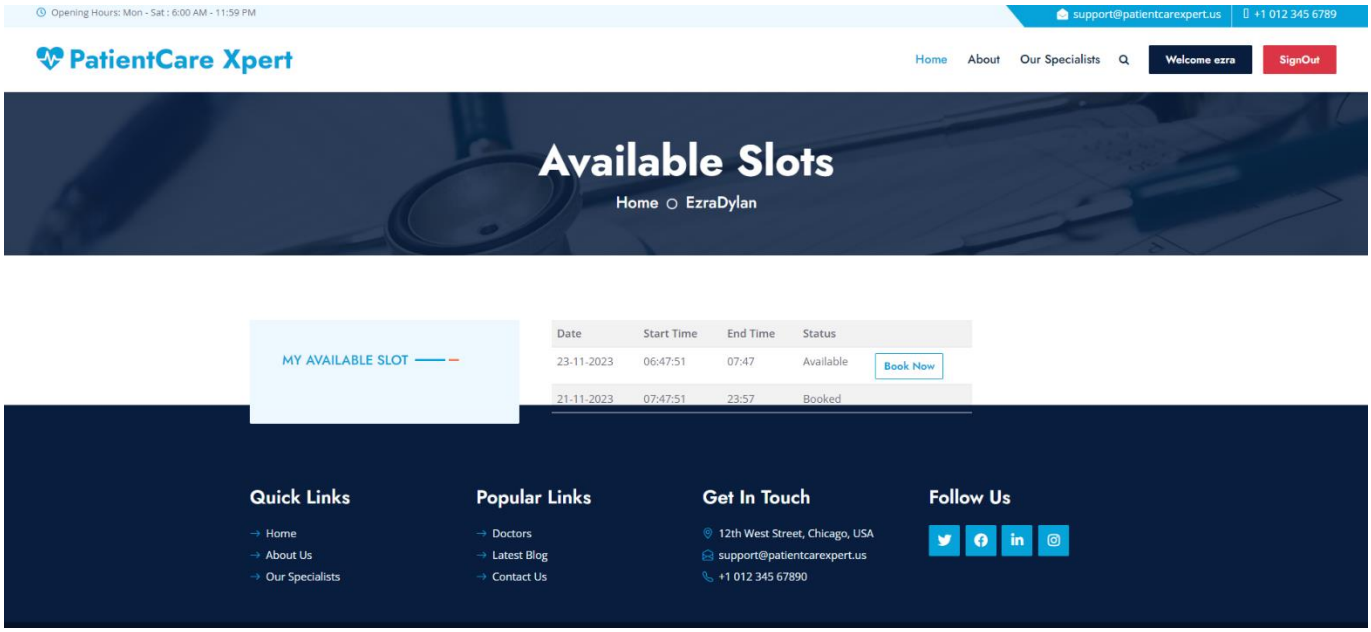


Figure 10 Available Slots Page

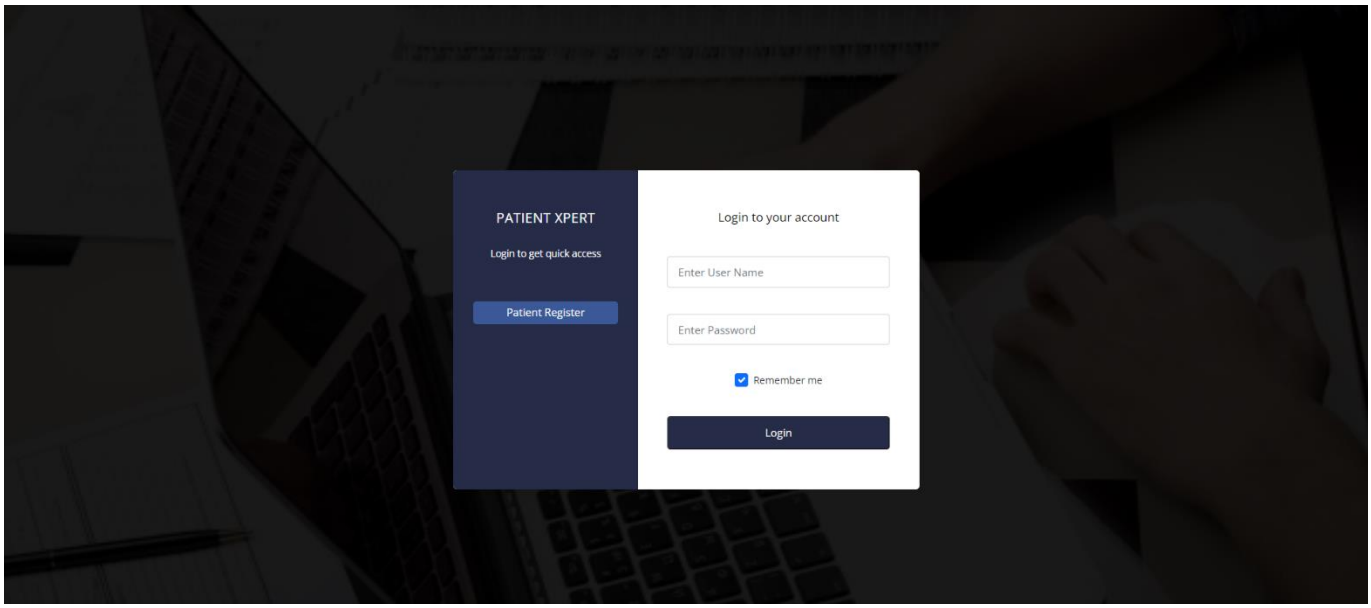


Figure 11 Login Page

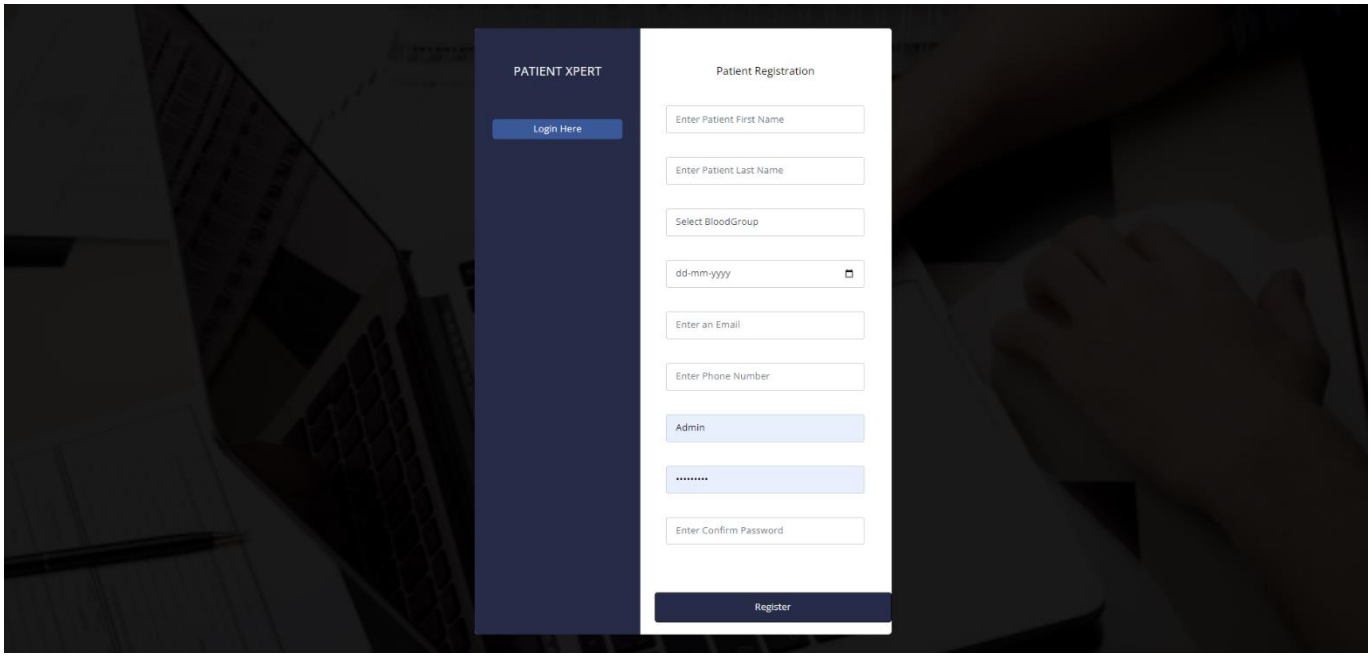


Figure 12 Registration Page

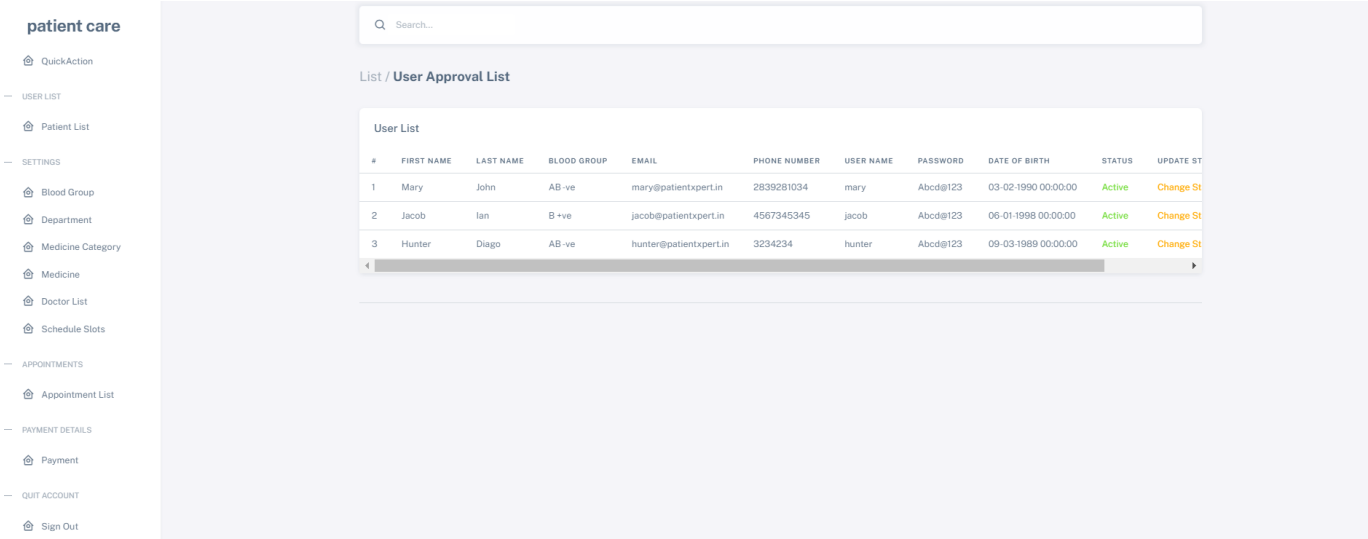


Figure 13 Patient List Page

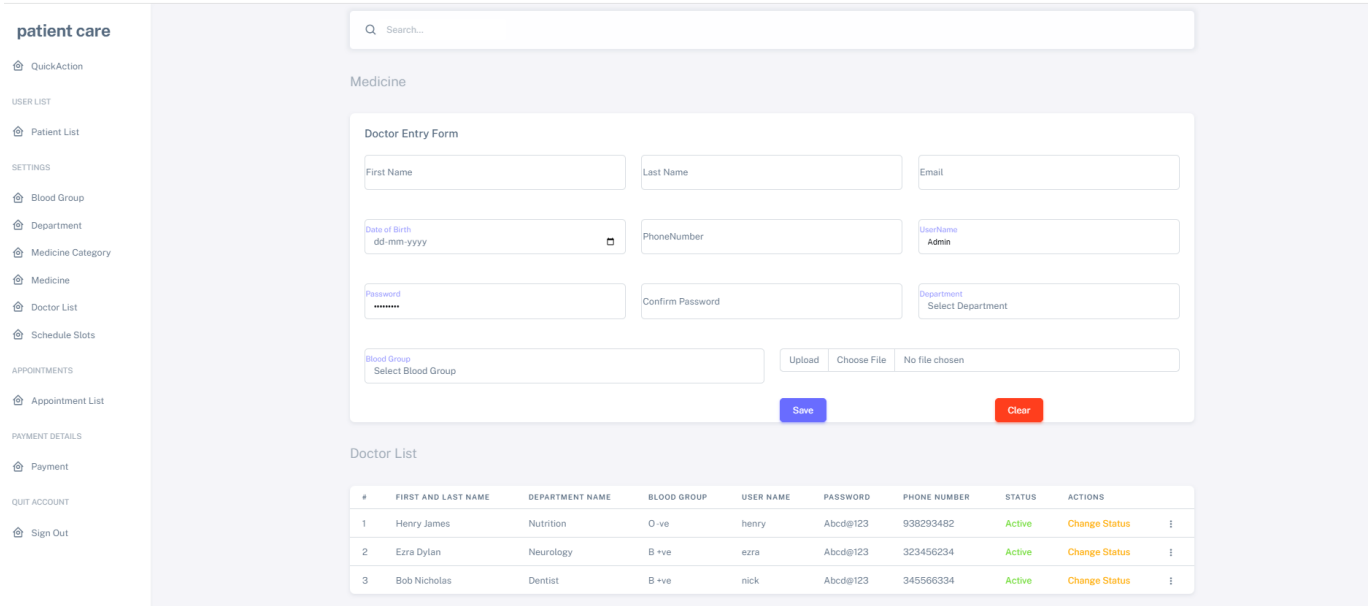


Figure 14 Doctor List Page

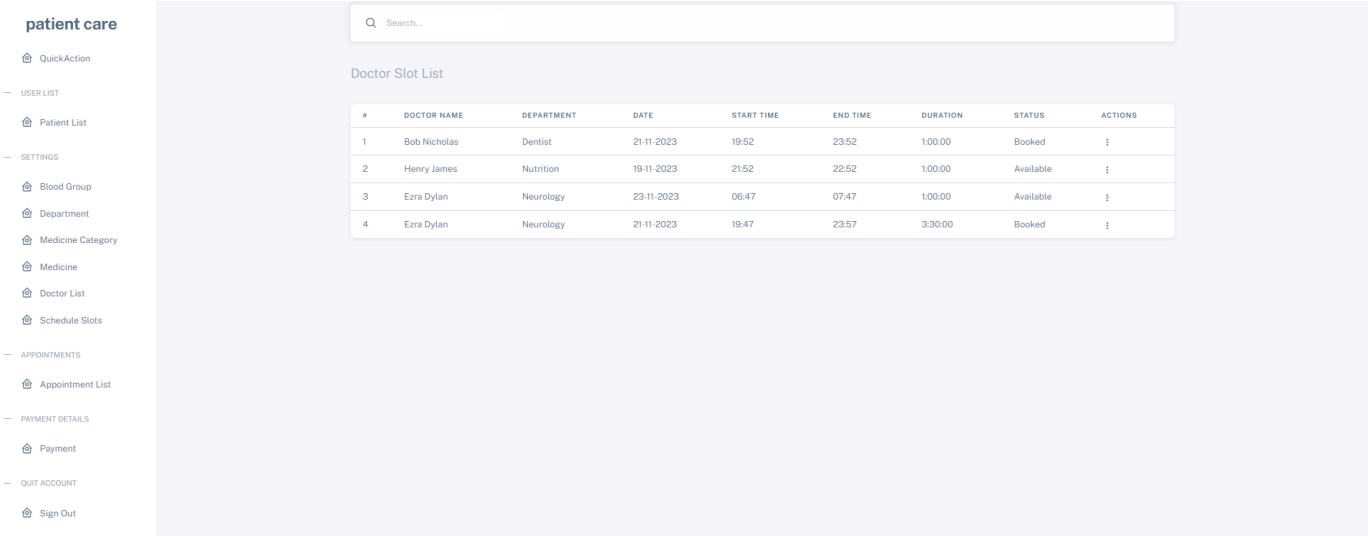


Figure 15 Doctor Slot List Page

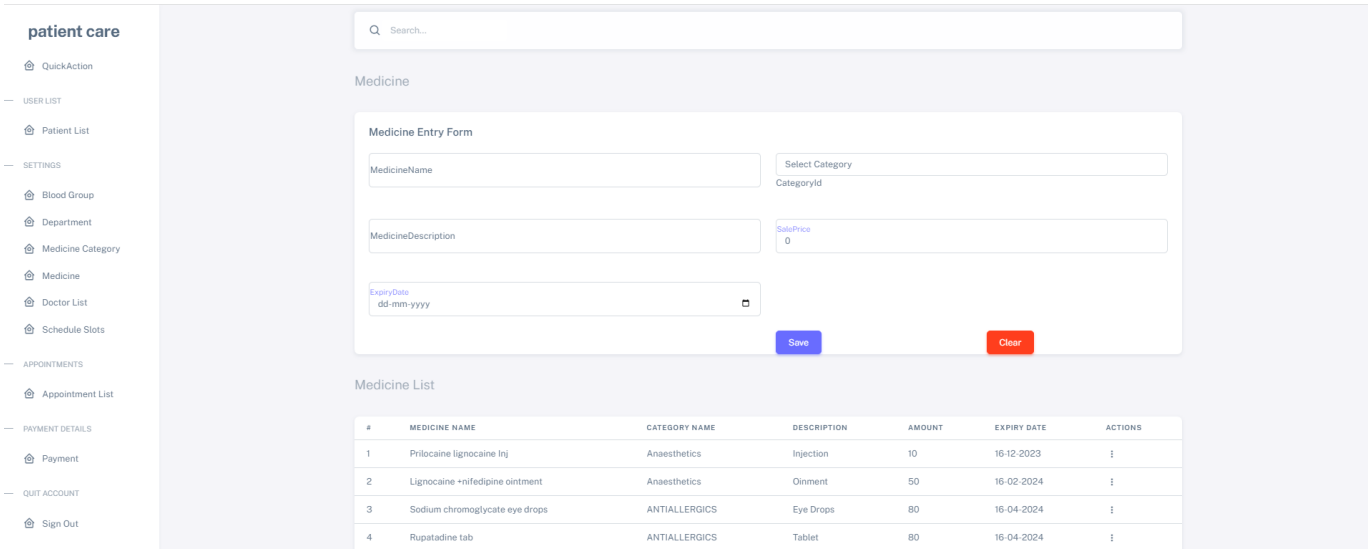


Figure 16 Medicine List Page

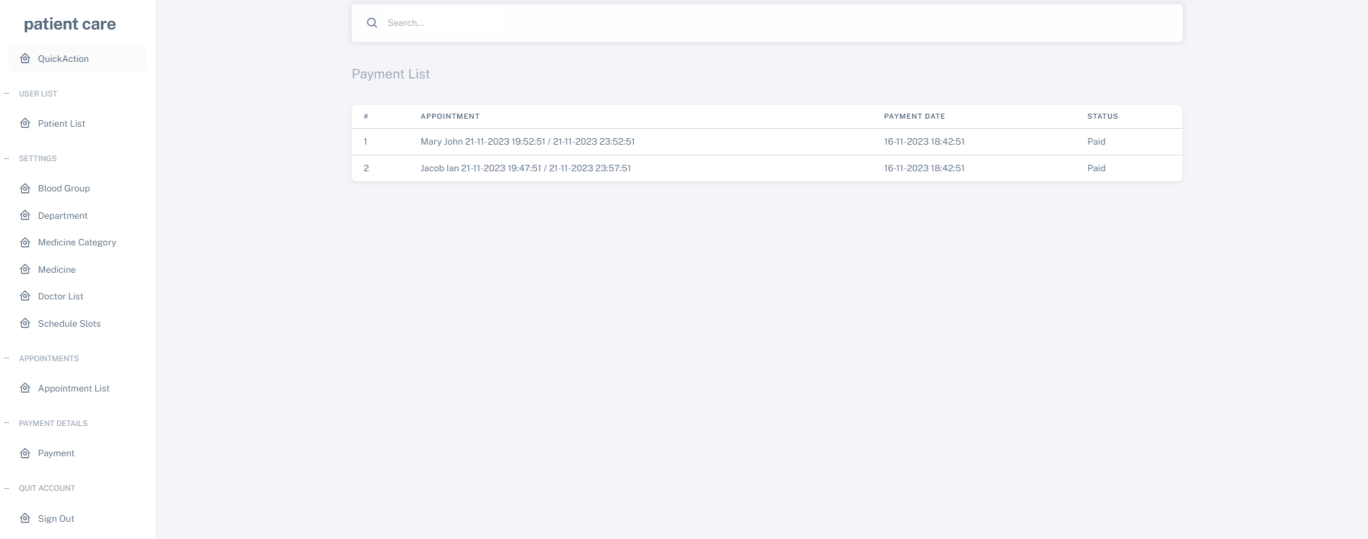


Figure 17 Payment List Page

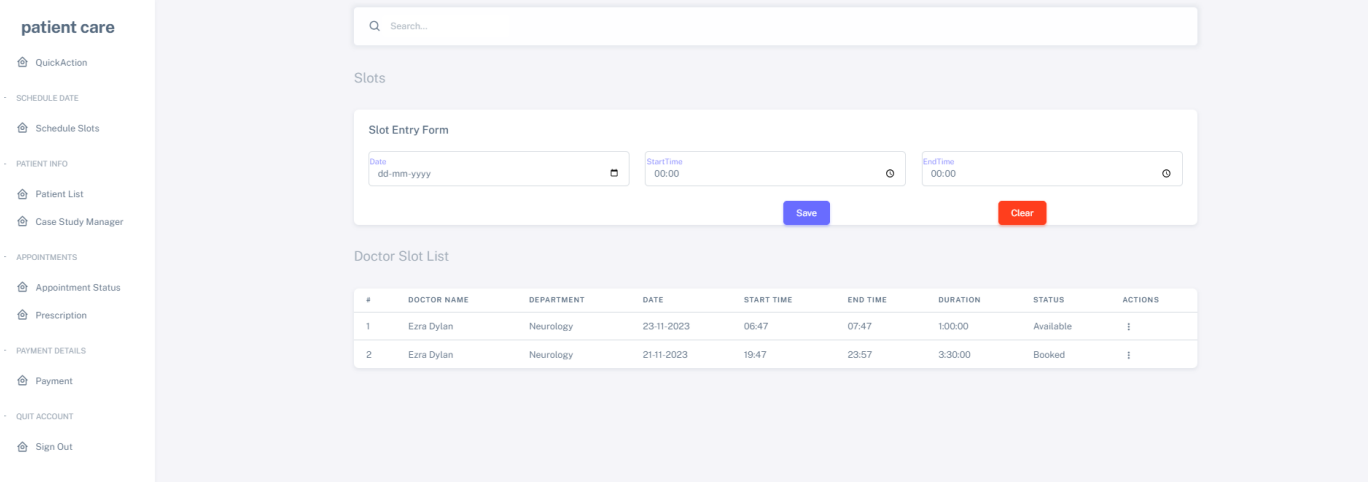


Figure 18 Doctor Slot List Page

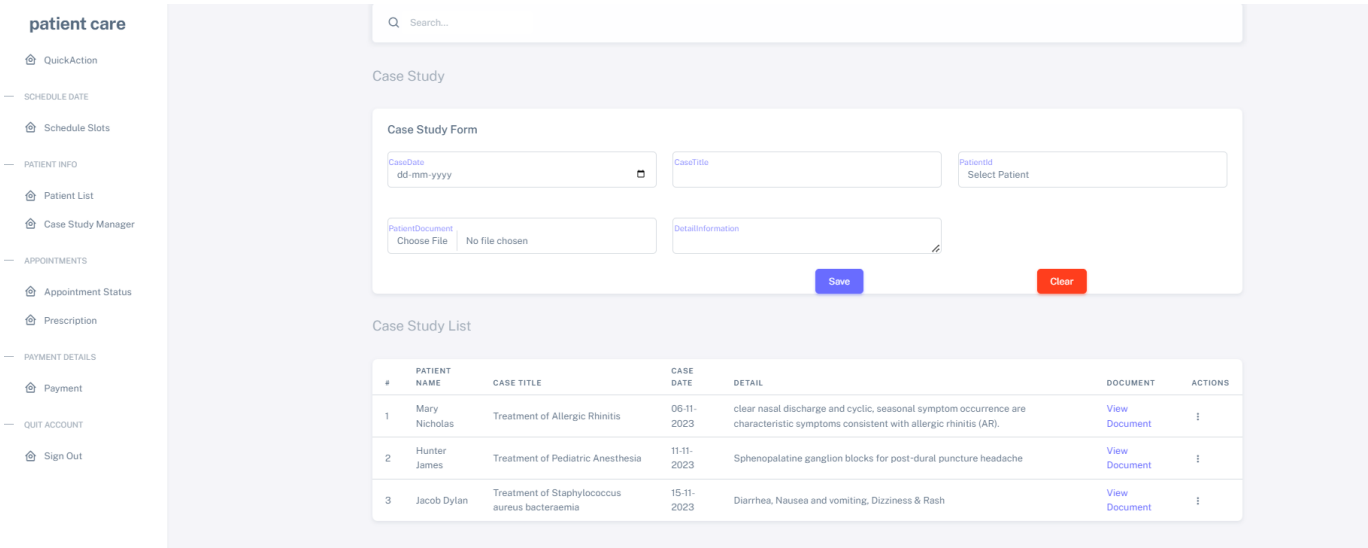


Figure 19 Case Study Page

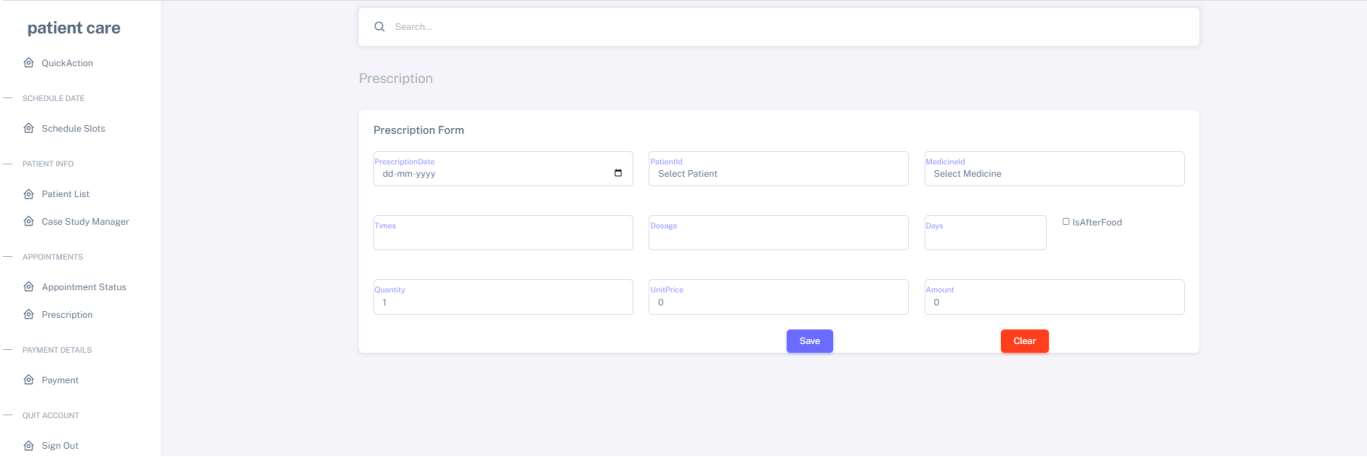


Figure 20 Prescription Form Page

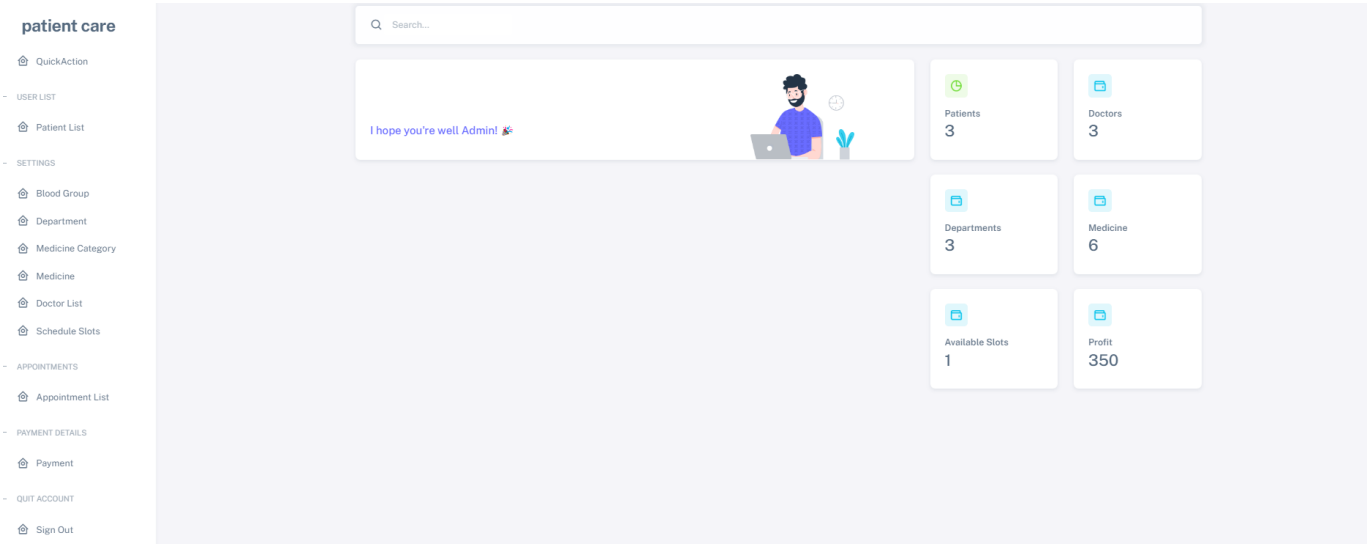


Figure 21 Admin Data Quick Action Page

6 *Design Units Impacts*

A website's or an application's design has a big impact on the business. Developers can use this design method to identify the needs of the company. The design process will be accessed in accordance with the requirements of the business, and the design approach may be altered to satisfy the client's requirements. The process of receiving input ought to be included in design. The software's design will benefit from client feedback, which will also help to protect the storage of customer data. The design goal should specify the strategy for maintaining security and spotting fraud.

6.1 Functional Area A/Design Unit A

6.1.1 Functional Overview

Design Unit A in PatientCare Xpert focuses on the crucial functional area of the appointment management system. This encompasses the end-to-end process of appointment scheduling, approval, and real-time updates. Patients can initiate appointment requests, doctors can manage their availability, and administrators oversee the entire appointment workflow. The system provides a centralized and user-friendly interface for seamless interactions among stakeholders.

6.1.2 Impacts

The impacts of Design Unit A are profound, influencing the overall efficiency and user experience of the appointment management process. It streamlines the workflow, reducing manual efforts in appointment coordination and approval. Patients benefit from a straightforward booking process, doctors can efficiently manage their schedules, and administrators gain a comprehensive overview of the appointment landscape. Real-time updates ensure that all stakeholders stay informed, contributing to a transparent and responsive healthcare management platform.

6.1.3 Requirements

To effectively implement Design Unit A, several requirements must be addressed. These include the development of a user-friendly interface for patients to initiate appointments, a robust system for doctors to manage their availability and approve appointments, and an administrative dashboard for overseeing the entire appointment workflow. Additionally, real-time communication channels and secure data storage are imperative to ensure the integrity and timely updates of the appointment management system. Compliance with healthcare data protection standards and the seamless integration of the appointment system with other platform functionalities are also essential requirements for Design Unit A.

7 Open Issues

As PatientCare Xpert progresses through development, a few open issues require attention. These issues include fine-tuning the real-time communication channels for video consultations, optimizing the responsiveness of the user interface across various devices, and conducting thorough testing to identify and address any potential security vulnerabilities. Additionally, the integration of external meeting links for remote consultations and the seamless synchronization of appointment updates across user dashboards are areas that need careful consideration. Open communication channels with the development team and stakeholders will be maintained to promptly address and resolve these issues to ensure the platform's reliability and user satisfaction.

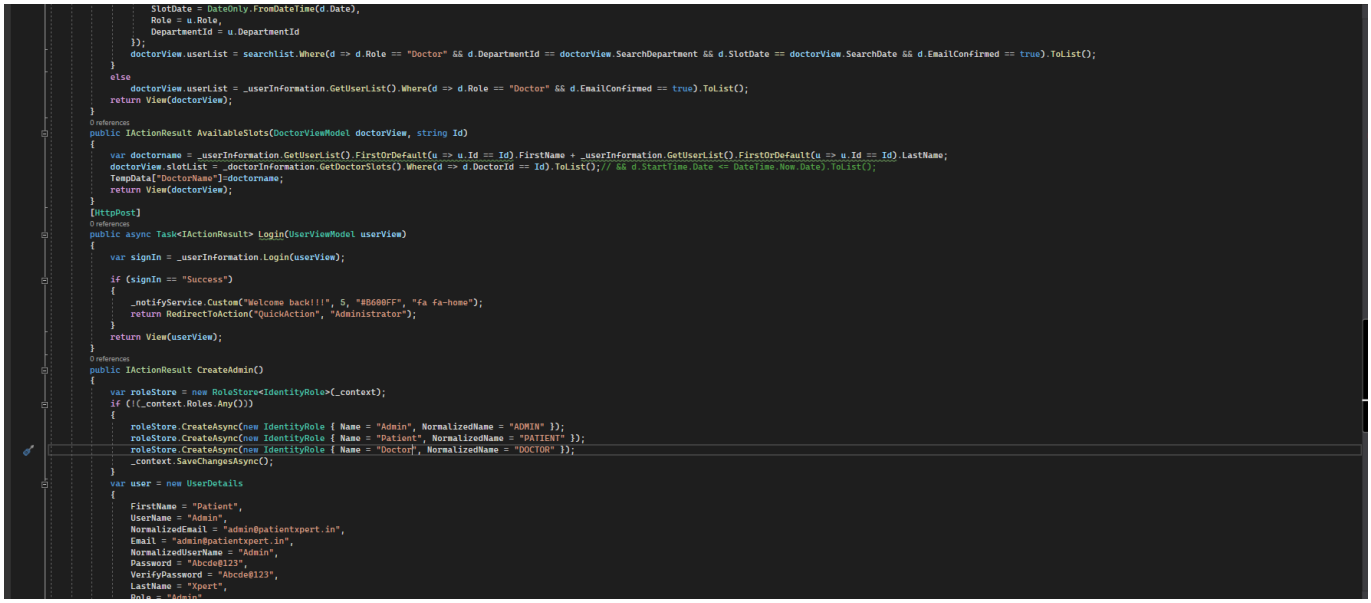
8 *Acknowledgements*

Thank you to everyone who has been part of this journey and has contributed to the success of PatientCare Xpert. We would like to express our sincere gratitude to our esteemed professor, Assistant Professor Xin Chen, for their unwavering guidance, mentorship, and invaluable support throughout the development of PatientCare Xpert. Their expertise, encouragement, and insightful feedback have played a pivotal role in shaping the success of this project.

9 *References*

1. 2010. "LINQ." In *Programming Microsoft®*, by Marco Russo & Paolo Pialorsi.
2. *ProHealthConnect*. https://www.prohealthconnect.com/.
3. *SignalR*. https://learn.microsoft.com/en-us/aspnet/core/tutorials/signalr?view=aspnetcore-8.0&tabs=visual-studio.

10 *Appendices*



```
                SlotDate = DateOnly.FromDateTime(d.Date),
                Role = u.Role,
                DepartmentId = u.DepartmentId
            });
            doctorView.userList = searchlist.Where(d => d.Role == "Doctor" && d.DepartmentId == doctorView.SearchDepartment && d.SlotDate == doctorView.SearchDate && d.EmailConfirmed == true).ToList();
        }
        else
            doctorView.userList = _userInformation.GetUserList().Where(d => d.Role == "Doctor" && d.EmailConfirmed == true).ToList();
        return View(doctorView);
    }
    0 references
    public IActionResult AvailableSlots(DoctorViewModel doctorView, string Id)
    {
        var doctorname = _userInformation.GetUserList().FirstOrDefault(u => u.Id == Id).FirstName + _userInformation.GetUserList().FirstOrDefault(u => u.Id == Id).LastName;
        doctorView.slotList = _doctorInformation.GetDoctorSlots().Where(d => d.DoctorId == Id).ToList();// && d.StartTime.Date <= DateTime.Now.Date).ToList();
        TempData["DoctorName"]=doctorname;
        return View(doctorView);
    }
    [HttpPost]
    0 references
    public async Task<IActionResult> Login(UserViewModel userView)
    {
        var signIn = _userInformation.Login(userView);

        if (signIn == "Success")
        {
            _notifyService.Custom("Welcome back!!!", 5, "#B600FF", "fa fa-home");
            return RedirectToAction("QuickAction", "Administrator");
        }
        return View(userView);
    }
    0 references
    public IActionResult CreateAdmin()
    {
        var roleStore = new RoleStore<IdentityRole>(_context);
        if (!(_context.Roles.Any()))
        {
            roleStore.CreateAsync(new IdentityRole { Name = "Admin", NormalizedName = "ADMIN" });
            roleStore.CreateAsync(new IdentityRole { Name = "Patient", NormalizedName = "PATIENT" });
            roleStore.CreateAsync(new IdentityRole { Name = "Doctor", NormalizedName = "DOCTOR" });
            _context.SaveChangesAsync();
        }
        var user = new UserDetails
        {
            FirstName = "Patient",
            UserName = "Admin",
            NormalizedEmail = "admin@patientxpert.in",
            Email = "admin@patientxpert.in",
            NormalizedUserName = "Admin",
            Password = "Abcde@123",
            VerifyPassword = "Abcde@123",
            LastName = "Xpert",
            Role = "Admin"
```

Figure 22 Home Controller Page

```
using Microsoft.AspNetCore.Mvc;
using Twilio;
using Twilio.Clients;
using Twilio.Rest.Proxy.V1.Service.Session;
using Twilio.Rest.Video.V1;
using Twilio.Rest.Video.V1.Room.Participant;

namespace PatientcareXpert.Controllers
{

    1 reference
    public class VideoController : Controller
    {
        readonly IVideoService _videoService;

        0 references
        public VideoController(IVideoService videoService)
            => _videoService = videoService;

        [HttpGet("token")]
        0 references
        public IActionResult GetToken()
            => new JsonResult(new { token = _videoService.GetTwilioJwt(User.Identity.Name) });

        [HttpGet("rooms")]
        0 references
        public async Task<IActionResult> GetRooms()
            => new JsonResult(await _videoService.GetAllRoomsAsync());
        0 references
        public IActionResult VideoChat()
        {
            return View();
        }
        //private readonly ITwilioRestClient _client;
```

Figure 23 Video Controller Page

```
@model PatientcareXpert.Models.ViewModel.UserViewModel
@{
    ViewData["Title"] = "Medicine";
    Layout = "_SecondaryLayout";
}
<div class="container-xxl flex-grow-1 container-p-y">
    <h4 class="fw-bold py-3 mb-4"><span class="text-muted fw-light">Medicine</span></h4>
    <div class="row">
        <form asp-controller="Administrator" asp-action="AddDoctor" method="post" enctype="multipart/form-data">
            <div class="col-md-12">
                <div class="card mb-4">
                    <h5 class="card-header">Doctor Entry Form</h5>
                    <div class="card-body row">
                        <div class="form-floating col-md-4">
                            @*<input type="hidden" asp-for="userDetails.Id" />*@
                            <input type="text"
                                   class="form-control" asp-for="userDetails.FirstName"
                                   placeholder="John"
                                   aria-describedby="floatingInputHelp" />
                            <label asp-for="userDetails.FirstName"></label>
                        </div>
                        <div class="form-floating col-md-4">
                            <input type="text"
                                   class="form-control" asp-for="userDetails.LastName"
                                   placeholder="N"
                                   aria-describedby="floatingInputHelp" />
                            <label asp-for="userDetails.LastName"></label>
                        </div>
                        <div class="form-floating col-md-4">
                            <input type="email"
                                   class="form-control" asp-for="userDetails.Email"
                                   placeholder="John@gmail.com"
                                   aria-describedby="floatingInputHelp" />
                            <label asp-for="userDetails.Email"></label>
                        </div>
                    </div>
                    <div class="card-body row">
                        <div class="form-floating col-md-4">
                            <input type="date"
                                   class="form-control" asp-for="userDetails.DOB"
                                   placeholder="N"
                                   aria-describedby="floatingInputHelp" />
                            <label asp-for="userDetails.DOB"></label>
                        </div>
                        <div class="form-floating col-md-4">
                            <input type="text"
                                   class="form-control" asp-for="userDetails.PhoneNumber"
                                   placeholder="7234234234"
                                   aria-describedby="floatingInputHelp" />
                            <label asp-for="userDetails.PhoneNumber"></label>
                        </div>
                        <div class="form-floating col-md-4">
                            <input type="text"
                                   class="form-control" asp-for="userDetails.UserName"
```

Figure 24 View Pages of Add Doctor

```
@model PatientCareXpert.Models.ViewModel.AdminViewModel
@{
    ViewData["Title"] = "Department";
    Layout = "_SecondaryLayout";
}
<div class="container-xxl flex-grow-1 container-p-y">
    <h4 class="fw-bold py-3 mb-4"><span class="text-muted fw-light">Department</span></h4>
    <div class="row">
        <form asp-controller="Administrator" asp-action="Department" method="post">
            <div class="col-md-12">
                <div class="card mb-4">
                    <h5 class="card-header">Department Entry Form</h5>
                    <div class="card-body row">
                        <div class="form-floating col-md-6">
                            <input type="hidden" asp-for="department.Id" />
                            <input type="text"
                                   class="form-control" asp-for="department.DName"
                                   placeholder="Nutrition"
                                   aria-describedby="floatingInputHelp" />
                            <label asp-for="department.DName"></label>
                        </div>
                        <div class="form-floating col-md-6">
                            <textarea type="text"
                                      class="form-control" asp-for="department.Description"
                                      placeholder="..........."
                                      aria-describedby="floatingInputHelp"></textarea>
                            <label asp-for="department.Description"></label>
                        </div>
                    </div>
                    <div class="row justify-content-end">
                        <div class="col-sm-10">
                            <button type="submit" class="btn btn-primary">Save</button>
                        </div>
                    </div>
                </div>
            </div>
        </form>
    </div>
    @if (Model.departmentList.Count > 0 && Model.departmentList != null)
    {
        <h4 class="fw-bold py-3 mb-4"><span class="text-muted fw-light">Department List</span></h4>
        <!-- Basic Bootstrap Table -->
        <div class="card">
            <table class="table">
                <thead>
                    <tr>
                        <th>#</th>
                        <th>Department</th>
                        <th>Description</th>
                        <th>Actions</th>
                    </tr>
                </thead>
                @{
                    int incre = 1;
```

Figure 25 Department View Page

```
@model PatientCareXpert.Models.ViewModel.AdminViewModel
@{
    ViewData["Title"] = "Medicine";
    Layout = "_SecondaryLayout";
}
<div class="container-xxl flex-grow-1 container-p-y">
    <h4 class="fw-bold py-3 mb-4"><span class="text-muted fw-light">Medicine</span></h4>
    <div class="row">
        <form asp-controller="Administrator" asp-action="Medicine" method="post">
            <div class="col-md-12">
                <div class="card mb-4">
                    <h5 class="card-header">Medicine Entry Form</h5>
                    <div class="card-body row">
                        <div class="form-floating col-md-6">
                            <input type="hidden" asp-for="medicine.Id" />
                            <input type="text"
                                   class="form-control" asp-for="medicine.MedicineName"
                                   placeholder="Plaster, Aspirin, Syringe"
                                   aria-describedby="floatingInputHelp" />
                            <label asp-for="medicine.MedicineName"></label>
                        </div>
                        <div class="col-md-6">
                            <select type="text"
                                    class="form-control" asp-for="medicine.CategoryId" asp-items="@Model.medicine.CategoryDrp">
                                <option>Select Category</option>
                            </select>
                            <label asp-for="medicine.CategoryId"></label>
                        </div>
                    </div>
                    <div class="card-body row">
                        <div class="form-floating col-md-6">
                            <input type="text"
                                   class="form-control" asp-for="medicine.MedicineDescription"
                                   placeholder="................"
                                   aria-describedby="floatingInputHelp" />
                            <label asp-for="medicine.MedicineDescription"></label>
                        </div>
                        <div class="form-floating col-md-6">
                            <input type="text"
                                   class="form-control" asp-for="medicine.SalePrice"
                                   placeholder="$.00"
                                   aria-describedby="floatingInputHelp" />
                            <label asp-for="medicine.SalePrice"></label>
                        </div>
                    </div>
                    <div class="card-body row">
                        <div class="form-floating col-md-6">
                            <input type="date"
                                   class="form-control" asp-for="medicine.ExpiryDate"
                                   placeholder="01-03-2024"
                                   aria-describedby="floatingInputHelp" value="@DateTime.Now.ToString("mm/dd/yyyy")"/>
                            <label asp-for="medicine.ExpiryDate"></label>
                        </div>
                    </div>
```

Figure 26 Medicine View Page